



UNIVERSIDADE FEDERAL DO ESPÍRITO SANTO
CENTRO TECNOLÓGICO
COLEGIADO DO CURSO DE CIÊNCIA DA COMPUTAÇÃO

Miguel Vieira Machado Pim

**Justiça de Grupo em Sistemas de
Recomendação: uma Análise de Agrupamentos
Latentes de Usuários**

Vitória, ES

2026

Miguel Vieira Machado Pim

Justiça de Grupo em Sistemas de Recomendação: uma Análise de Agrupamentos Latentes de Usuários

Anteprojeto apresentado ao Colegiado do
Curso de Ciência da Computação do Cen-
tro Tecnológico da Universidade Federal do
Espírito Santo, como requisito para aprova-
ção na Disciplina Projeto de Graduação I.

Universidade Federal do Espírito Santo – UFES

Centro Tecnológico

Colegiado do Curso de Ciência da Computação

Orientador: Prof. Dr. Diego Roberto Colombo Dias

Coorientador: Prof. Athus Cavalini

Vitória, ES

2026

1 Introdução

Nos últimos anos, a transformação digital tem revolucionado a maneira como interagimos com o mundo à nossa volta. Os sistemas digitais deixaram de oferecer interfaces rígidas e fluxos de interação uniformes e passaram a proporcionar interações mais dinâmicas e personalizadas aos usuários, que agora podem ter acesso a itens personalizados e uma experiência muito mais atrativa para os seus interesses. Nesse contexto, destacam-se os algoritmos de recomendação, que têm o papel de compreender as preferências dos usuários e recomendar novos itens alinhados às suas preferências.

Sistemas de recomendação são ferramentas e técnicas de software que fornecem sugestões de itens a serem utilizados por um usuário, atuando como mecanismos de filtragem de informação para mitigar o problema da sobrecarga de dados (RICCI; ROKACH; SHAPIRA, 2022). Em um cenário que as plataformas digitais têm se tornado cada vez mais interativas e personalizadas, os algoritmos de recomendação se tornam uma ferramenta essencial, capaz de moldar a experiência e o comportamento de consumo dos usuários. Quanto mais informações estão disponíveis sobre o usuário, maior tende a ser a qualidade dos itens recomendados (HAZRATI; RICCI, 2024; CAVALCANTE; FETTERMANN, 2019; PEREIRA et al., 2018; BERNARDINO; GONÇALVES, 2019).

À medida que os os sistemas de recomendação exercem um impacto crescente na tomada de decisão dos indivíduos, surgem preocupações sobre seu potencial de causar, ainda que inadvertidamente, danos sociais. Estudos recentes mostram que sistemas de recomendação podem oferecer uma qualidade de serviço injusta ou desigual para um usuário ou para grupos de usuários (BEUTEL et al., 2017; BURKE; SONBOLI; ORDONEZ-GAUGER, 2018), ou podem levar à polarização social por aumentarem a divergência entre preferências individuais ou de grupos de usuários (DANDEKAR; GOEL; LEE, 2013).

Diante deste contexto, este trabalho propõe a integração entre métricas de justiça de grupo e algoritmos de recomendação com o objetivo de identificar desigualdades na qualidade das recomendações recebidas por diferentes grupos de usuários. Para isso, pretende-se analisar não apenas agrupamentos definidos por características observáveis, mas também grupos latentes obtidos por meio de técnicas de clusterização, permitindo investigar se determinadas estruturas de usuários apresentam maior desvantagem nas recomendações geradas.

Além disso, este trabalho busca estender análises anteriores (SANTOS; COMARELA, 2025) ao considerar bases de dados maiores e modelos de recomendação baseados em redes neurais, capazes de capturar relações mais complexas entre usuários e itens. Com isso, espera-se contribuir para uma compreensão mais ampla sobre como diferentes

estratégias de recomendação afetam a justiça entre grupos, fornecendo evidências que possam apoiar o desenvolvimento de sistemas de recomendação mais justos, inclusivos e adequados a cenários reais.

1.1 Motivação e Justificativa

A principal motivação para este trabalho reside no fato de que algoritmos de recomendação podem incorporar ou reproduzir vieses presentes nos dados durante a modelagem e, conseqüentemente, gerar recomendações injustas para determinados grupos de usuários. Por exemplo, um sistema de recomendação de vagas de emprego pode apresentar discriminação de gênero ao recomendar vagas com menor remuneração para determinados grupos de usuários (DATTA; TSCHANTZ; DATTA, 2015). Esse tipo de problema evidencia a necessidade de estudos capazes de identificar grupos para os quais o algoritmo apresenta maior nível de injustiça, contribuindo para o desenvolvimento de sistemas de recomendação mais justos e inclusivos.

Trabalhos anteriores investigaram essa questão utilizando algoritmos baseados em fatoração de matriz e análise de vizinhança de usuários, como *Alternating Least Squares* (ALS), *Non-negative Matrix Factorization* (NMF) e *K-Nearest Neighbors* (KNN), além de utilizarem conjuntos de dados reduzidos ou fortemente filtrados (SANTOS; COMARELA, 2025). Embora esses estudos sejam importantes para compreender a presença de injustiça de grupo em sistemas de recomendação, ainda há espaço para avaliar se os mesmos padrões de desigualdade permanecem quando são considerados modelos mais complexos e bases de dados maiores.

Portanto, este trabalho justifica-se pela necessidade de estender essas análises para algoritmos de recomendação baseados em redes neurais, que podem capturar relações mais complexas e não lineares entre usuários e itens, e para conjuntos de dados maiores, mais próximos de cenários reais. Com isso, espera-se verificar se grupos definidos por atributos observáveis ou por técnicas de clusterização continuam apresentando diferenças relevantes na qualidade das recomendações recebidas.

1.2 Objetivos

1.2.1 Objetivo Geral

Analisar a justiça de grupo em sistemas de recomendação baseados em redes neurais, investigando se agrupamentos não triviais de usuários permitem identificar grupos mais prejudicados pela qualidade das recomendações.

1.2.2 Objetivos Específicos

- Desenvolver um fluxo de pré-processamento dos conjuntos de dados para possibilitar o treinamento e a avaliação dos modelos de recomendação.
- Implementar e avaliar modelos de recomendação baseados em redes neurais.
- Realizar diferentes agrupamentos de usuários utilizando atributos observáveis disponíveis nos conjuntos de dados e técnicas de clusterização.
- Calcular métricas de justiça de grupo para diferentes agrupamentos de usuários, a fim de identificar grupos com maiores níveis de injustiça nas recomendações.
- Medir a qualidade das recomendações geradas pelos modelos por meio de métricas de erro, como *Root Mean Square Error* (RMSE).
- Comparar os resultados obtidos entre diferentes modelos de recomendação e estratégias de agrupamento, verificando em quais cenários surgem maiores desigualdades entre grupos.

1.3 Método de Desenvolvimento do Trabalho

A metodologia deste trabalho será organizada em etapas sequenciais, envolvendo a seleção e o pré-processamento dos conjuntos de dados, o treinamento de modelos de recomendação baseados em redes neurais, a definição de agrupamentos de usuários e a avaliação dos resultados por meio de métricas de qualidade preditiva e de justiça de grupo.

1.3.1 Dados

Este estudo utilizará dois conjuntos de dados públicos amplamente utilizados em pesquisas sobre sistemas de recomendação:

- **LastFM-360K**: Conjunto de dados composto por interações de *feedback* implícito que representam a quantidade de vezes que um usuário reproduziu músicas de um determinado artista (CELMA, 2010).
- **Yelp Open Dataset**¹: O *Yelp Open Dataset* é um conjunto de dados disponibilizado para fins educacionais e acadêmicos, contendo informações reais sobre estabelecimentos, como avaliações, fotos, *check-ins* e atributos relacionados a horário de funcionamento, estacionamento e ambiente.

¹ <<https://business.yelp.com/data/resources/open-dataset/>>

Após a coleta dos dados, será construída uma matriz usuário-item para cada conjunto de dados. No **LastFM-360K**, as interações serão derivadas da quantidade de execuções entre usuários e artistas e serão normalizadas para uma escala de 1 a 5, para que a métrica de justiça de grupo possa ser calculada. No **Yelp Open Dataset**, serão utilizadas as avaliações dos usuários sobre os estabelecimentos. Em seguida, serão aplicadas etapas de filtragem, tratamento de valores ausentes e divisão dos dados em conjuntos de treinamento, validação e teste. As etapas de filtragem poderão reduzir as dimensões originais dos conjuntos de dados, com o objetivo de garantir viabilidade computacional e densidade mínima de interações por usuário e item. A Tabela 1 mostra as dimensões originais dos conjuntos de dados.

	Usuários	Itens	Interações
LastFM-360K	359.347	294.015	17.559.530
Yelp Open Dataset	1.987.929	150.346	6.990.280

Tabela 1 – Estatísticas dos *datasets* utilizados

1.3.2 Estratégias de Recomendação

Após a etapa de preparação dos dados, serão treinados e avaliados modelos de recomendação baseados em redes neurais. Para essa etapa, será utilizada a biblioteca RecBole², que fornece implementações padronizadas de modelos de recomendação e ferramentas para carregamento dos dados, treinamento, validação e avaliação experimental.

Os modelos utilizados neste trabalho serão o *Neural Collaborative Filtering* (NeuMF) e o *Variational Autoencoders for Collaborative Filtering* (Mult-VAE). Esses modelos foram escolhidos por representarem abordagens neurais distintas para recomendação: o NeuMF utiliza redes neurais para modelar interações entre usuários e itens, enquanto o Mult-VAE utiliza uma arquitetura baseada em autoencoders variacionais para aprender representações latentes dos usuários.

1.3.3 Otimização de Hiperparâmetros

Os hiperparâmetros dos modelos serão definidos com base em configurações recomendadas na literatura e na documentação da biblioteca utilizada. Quando necessário, serão realizados testes preliminares com um conjunto reduzido de configurações, considerando a viabilidade computacional do projeto. Essa etapa tem como objetivo obter configurações adequadas para os modelos sem tornar o custo experimental inviável.

² <<https://recbole.io/>>

1.3.4 Clusterização de Usuários

Os usuários serão divididos em grupos a partir de atributos disponíveis nos conjuntos de dados e por meio de técnicas de clusterização. Os atributos utilizados para agrupamento poderão variar de acordo com a disponibilidade de metadados em cada conjunto de dados. Quando determinados atributos, como gênero ou idade, não estiverem disponíveis, serão utilizados atributos alternativos presentes na base, como localização, categoria de item ou nível de atividade do usuário.

Entre os agrupamentos considerados, será analisado o nível de atividade dos usuários, separando os usuários mais ativos dos demais com base na quantidade de interações realizadas. Além disso, serão aplicadas técnicas de clusterização, como a clusterização aglomerativa, com o objetivo de identificar grupos latentes de usuários. A quantidade de grupos será definida a partir do hiperparâmetro k , escolhido com base no coeficiente de silhueta.

1.3.5 Análise de Métricas

Na etapa final, os modelos serão avaliados por meio da métrica RMSE, utilizada para medir o erro preditivo das recomendações, e por métricas de justiça de grupo, utilizadas para verificar diferenças na qualidade das recomendações entre os grupos de usuários. O objetivo dessa análise é identificar grupos mais prejudicados pelas recomendações e verificar se os agrupamentos obtidos por técnicas de clusterização revelam desigualdades que não seriam observadas apenas por divisões diretas baseadas em metadados.

A partir desses resultados, serão comparadas as combinações entre modelos, conjuntos de dados e estratégias de agrupamento, com o objetivo de identificar quais cenários apresentam maiores níveis de injustiça de grupo.

1.4 Cronograma

A execução do projeto será distribuída ao longo de cinco meses, conforme as atividades descritas a seguir e organizadas no cronograma.

- **Atividade 1 - Pré-processamento de dados:** realizar o carregamento dos conjuntos de dados e implementar as etapas de pré-processamento, incluindo filtragem, correção de tipos, tratamento de valores ausentes e construção das matrizes usuário-item.
- **Atividade 2 - Implementação dos modelos:** configurar, treinar e validar os modelos de recomendação baseados em redes neurais utilizando a biblioteca escolhida para os experimentos.

- **Atividade 3 - Agrupamento de usuários:** definir agrupamentos de usuários a partir de atributos disponíveis nos conjuntos de dados e aplicar técnicas de clusterização para identificação de grupos latentes.
- **Atividade 4 - Ajuste de hiperparâmetros:** definir e testar configurações de hiperparâmetros para os modelos de recomendação e para os algoritmos de agrupamento, considerando a viabilidade computacional dos experimentos.
- **Atividade 5 - Avaliação dos resultados:** calcular métricas de qualidade preditiva, como RMSE, e métricas de justiça de grupo, comparando os resultados entre modelos, conjuntos de dados e estratégias de agrupamento.
- **Atividade 6 - Artigo:** desenvolvimento da escrita do artigo final, descrevendo todas as etapas anteriores.

	Ago/26	Set/26	Out/26	Nov/26	Dez/26
Atividade 1	X	X			
Atividade 2		X	X		
Atividade 3			X	X	
Atividade 4			X	X	
Atividade 5				X	X
Atividade 6	X	X	X	X	X

2 Fundamentação Teórica e Tecnologias Utilizadas

Este capítulo detalha os fundamentos teóricos e todas as tecnologias utilizadas para o desenvolvimento deste trabalho. A execução do projeto exige conhecimentos que abracem algoritmos de recomendação e clusterização, métricas de avaliação de justiça e qualidade da recomendação, assim como técnicas de pré-processamento de dados.

2.1 Algoritmos de Recomendação

Os algoritmos de recomendação utilizados neste trabalho são modelos generativos de recomendação que utilizam apenas uma matriz de interação usuários-item para fazer as recomendações.

2.1.1 Neural Matrix Factorization

O NeuMF, proposto por (HE et al., 2017), é um modelo de filtragem colaborativa que utiliza redes neurais para aprender uma função de interação entre usuários e itens. Sua principal motivação é combinar a capacidade da fatoração de matrizes de representar interações entre fatores latentes com a capacidade das redes neurais profundas de aprender relações não lineares entre usuários e itens.

A Figura 1 apresenta a arquitetura do NeuMF, composta por dois componentes paralelos: o *Generalized Matrix Factorization* (GMF) e o *Multi-Layer Perceptron* (MLP). Cada componente utiliza *embeddings* de usuários e itens e produz uma representação da interação entre eles. Ao final, as representações produzidas pelo GMF e pelo MLP são combinadas para calcular o valor estimado $\hat{y}_{ui}$ para o par formado pelo usuário u e pelo item i .

2.1.1.1 Camadas de entrada e de *embeddings*

A entrada do modelo é formada por dois vetores esparsos, $\mathbf{v}_u^U$ e $\mathbf{v}_i^I$, que identificam, respectivamente, o usuário u e o item i . As camadas de embeddings transformam essas representações esparsas em vetores densos de menor dimensionalidade. Esses embeddings podem ser interpretados como representações latentes dos usuários e dos itens, aprendidas a partir dos padrões presentes nas interações observadas.

De maneira geral, os embeddings de um usuário e de um item podem ser representados por

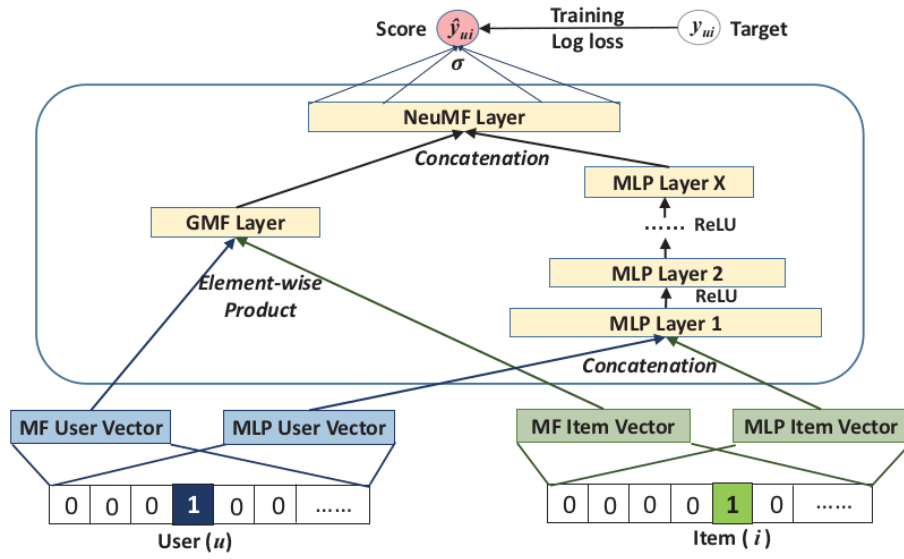


Figura 1 – Arquitetura NeuMF (HE et al., 2017)

$$\begin{aligned} \mathbf{p}_u &= \mathbf{P}^T \mathbf{v}_u^U, \\ \mathbf{q}_i &= \mathbf{Q}^T \mathbf{v}_i^I, \end{aligned} \quad (2.1)$$

em que $\mathbf{P}$ e $\mathbf{Q}$ correspondem às matrizes de embeddings dos usuários e dos itens, respectivamente. No NeuMF, os componentes GMF e MLP utilizam conjuntos independentes de embeddings, permitindo que cada componente aprenda representações adequadas à sua própria função de interação.

2.1.1.2 Camada GMF

A camada GMF representa uma generalização da função de interação utilizada na fatoração de matrizes tradicional. O GMF recebe os embeddings do usuário e do item, representados por $\mathbf{p}_u$ e $\mathbf{q}_i$, respectivamente. A interação entre esses vetores é calculada por meio do produto elemento a elemento:

$$\phi_1(\mathbf{p}_u, \mathbf{q}_i) = \mathbf{p}_u \odot \mathbf{q}_i. \quad (2.2)$$

Em seguida, esse vetor resultante é projetado para a camada de saída do modelo através da seguinte equação:

$$\hat{y}_{ui} = a_{out} \left(\mathbf{h}^T (\mathbf{p}_u \odot \mathbf{q}_i) \right), \quad (2.3)$$

onde a_{out} representa a função de ativação da camada de saída e h denota os pesos das arestas conectados a ela.

2.1.1.3 Camada MLP

O componente MLP é incorporado ao NeuMF com o objetivo de capturar relações complexas e não lineares entre os *embeddings* de usuários e itens. Diferentemente do GMF, que combina os vetores latentes por meio do produto elemento a elemento, o MLP concatena os *embeddings* do usuário e do item e os processa por meio de camadas totalmente conectadas. Dessa forma, a rede passa a aprender uma função de interação mais flexível entre usuários e itens, em vez de depender apenas de uma operação fixa entre os vetores latentes. Formalmente a estrutura de processamento das camadas da rede MLP é definida pela Equação 2.4.

$$\begin{aligned}
 \mathbf{z}_1 &= \phi_1(\mathbf{p}_u, \mathbf{q}_i) = \begin{bmatrix} \mathbf{p}_u \\ \mathbf{q}_i \end{bmatrix} \\
 \phi_2(\mathbf{z}_1) &= a_2(\mathbf{W}_2^T \mathbf{z}_1 + \mathbf{b}_2) \\
 &\vdots \\
 \phi_L(\mathbf{z}_{L-1}) &= a_L(\mathbf{W}_L^T \mathbf{z}_{L-1} + \mathbf{b}_L) \\
 \hat{y}_{ui} &= \sigma(\mathbf{h}^T \phi(\mathbf{z}_{L-1})),
 \end{aligned} \tag{2.4}$$

em que p_u e q_i representam, respectivamente, os embeddings do usuário u e do item i utilizados pelo componente MLP. Além disso, W_x , b_x e a_x denotam, respectivamente, a matriz de pesos, o vetor de vies e a função de ativação da x -ésima camada da rede. Outro aspecto relevante do MLP é a escolha da função de ativação utilizada em suas camadas ocultas. Embora funções como sigmoide e tangente hiperbólica possam ser utilizadas, o modelo NeuMF adota a função *Rectified Linear Unit* (ReLU). Essa função é adequada para o contexto de recomendação porque é menos suscetível à saturação (GLOROT; BORDES; BENGIO, 2011) e favorece ativações esparsas, característica importante em bases de dados de recomendação, que normalmente apresentam matrizes usuário-item bastante esparsas. Além disso, ela incentiva ativações esparsas, sendo adequada para dados esparsos e tornando o modelo menos propenso ao *overfitting*.

Quanto à arquitetura da rede, uma escolha comum é utilizar uma estrutura em torre, na qual a camada inicial possui maior número de neurônios e as camadas posteriores reduzem gradualmente essa dimensionalidade. A ideia dessa organização é permitir que as primeiras camadas capturem combinações mais amplas entre os *embeddings*, enquanto as camadas mais profundas aprendem representações progressivamente mais abstratas e compactas das interações usuário-item.

2.1.1.4 Fusão das camadas

A fusão entre GMF e MLP é o elemento central do NeuMF. Essa combinação busca aproveitar características complementares dos dois componentes: o GMF preserva

a capacidade da fatoração de matrizes de modelar interações latentes por meio de uma operação próxima ao produto interno, enquanto o MLP permite aprender relações não lineares mais complexas entre usuários e itens.

Uma possibilidade seria compartilhar a mesma camada de *embeddings* entre GMF e MLP, uma abordagem bem similar ao *Neural Tensor Network* (NTN) (SOCHER et al., 2013). No entanto, essa estratégia pode limitar o modelo, pois obriga os dois componentes a utilizarem representações latentes com a mesma dimensionalidade e com os mesmos parâmetros. Como cada componente possui uma forma distinta de modelar a interação usuário-item, o NeuMF utiliza embeddings separados para o GMF e para o MLP, permitindo que cada parte aprenda representações mais adequadas à sua própria função de interação.

Assim, o componente GMF recebe os embeddings p_u^G e q_i^G , enquanto o componente MLP recebe os embeddings p_u^M e q_i^M . A saída do GMF é obtida pelo produto elemento a elemento entre os embeddings correspondentes. Já a saída do MLP corresponde à última representação oculta produzida pela rede. Em seguida, essas duas representações são concatenadas e projetadas para a camada de saída, responsável por produzir a estimativa $\hat{y}_{ui}$ para o par usuário-item. Esse processo é descrito pela Equação 2.5.

$$\begin{aligned}\phi^{GMF} &= \mathbf{p}_u^G \odot \mathbf{q}_i^G, \\ \phi^{MLP} &= a_L(\mathbf{W}_L^T(a_{L-1}(\dots a_2(\mathbf{W}_2^T \begin{bmatrix} \mathbf{p}_u^M \\ \mathbf{q}_i^M \end{bmatrix} + \mathbf{b}_2)\dots)) + \mathbf{b}_L), \\ \hat{y}_{ui} &= \sigma \left(\mathbf{h}^T \begin{bmatrix} \phi^{GMF} \\ \phi^{MLP} \end{bmatrix} \right),\end{aligned}$$

em que $\mathbf{p}_u^G$ e $\mathbf{q}_i^G$ representam os *embeddings* de usuário e item utilizados pelo componente GMF, enquanto $\mathbf{p}_u^M$ e $\mathbf{q}_i^M$ representam os *embeddings* utilizados pelo componente MLP. O símbolo $\odot$ indica o produto elemento a elemento, $\mathbf{h}$ representa o vetor de pesos da camada de saída e σ corresponde à função de ativação utilizada nessa camada.

Dessa forma, o NeuMF combina a capacidade do GMF de capturar interações lineares entre fatores latentes com a capacidade do MLP de aprender padrões não lineares. Essa arquitetura torna o modelo mais expressivo do que a fatoração de matrizes tradicional, pois permite que a função de interação entre usuários e itens seja aprendida a partir dos dados. Na formulação original do NeuMF, a camada de saída utiliza uma função sigmoide para produzir valores adequados ao cenário de feedback implícito. Em adaptações para problemas de predição de avaliações, a função de saída e a função de perda podem ser ajustadas de acordo com a escala de valores adotada no experimento.

2.1.2 Variational Autoencoders for Collaborative Filtering

O Mult-VAE, proposto por (LIANG et al., 2018), é um algoritmo de recomendação baseado em *autoencoders variacionais*, desenvolvido com o objetivo de aplicar modelos probabilísticos não lineares ao problema de filtragem colaborativa. Um *autoencoder* é uma rede neural treinada para comprimir os dados de entrada em uma representação reduzida, chamada de espaço latente, preservando suas informações mais importantes. Em seguida, o modelo tenta reconstruir a entrada original a partir dessa representação, aprendendo padrões essenciais dos dados de forma não supervisionada. Diferentemente do *autoencoder* tradicional, o *autoencoder variacional* aprende os parâmetros de uma distribuição no espaço latente, estimando μ e σ para amostrar z , como ilustrado na Figura 2. Dessa forma, o modelo representa a incerteza dos padrões aprendidos antes de reconstruir a entrada pelo decodificador.

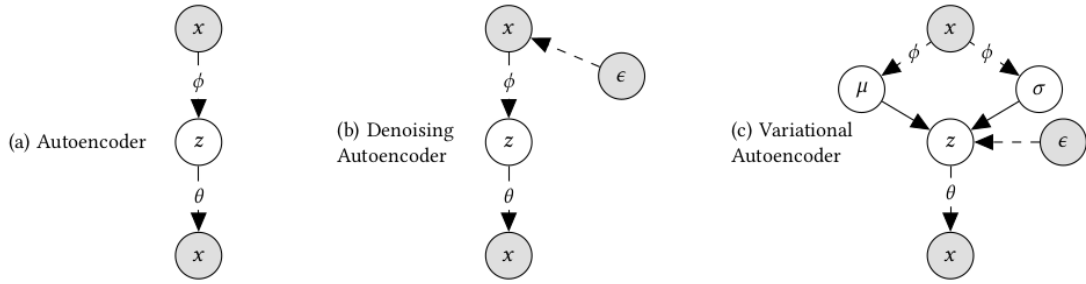


Figura 2 – Taxonomia de *autoencoders*. As setas pontilhadas representam operações de amostragem (LIANG et al., 2018)

A arquitetura do Mult-VAE é composta por dois componentes principais: o codificador e o decodificador. Assim, nas próximas seções, serão descritos a representação da entrada do modelo, o funcionamento do codificador e o processo de reconstrução realizado pelo decodificador.

2.1.2.1 Entrada do Modelo

A entrada do modelo é construída a partir da matriz de interações usuário-item. Seja $X \in \mathbb{R}^{U \times I}$ uma matriz de interações usuário-item com U usuários e I itens, em que cada elemento x_{ui} representa a interação ou preferência observada do usuário u em relação ao item i . Dessa forma, cada usuário é representado por uma linha da matriz X , dada por:

$$\mathbf{x}_u = [x_{u1}, x_{u2}, \dots, x_{uI}]^T, \quad (2.5)$$

O vetor $\mathbf{x}_u$ representa o histórico de interações do usuário u com todos os itens do conjunto de dados. Em geral, esse vetor é esparso, pois cada usuário interage apenas com uma pequena parte dos itens disponíveis.

2.1.2.2 Codificador

O codificador, também chamado de modelo de inferência, recebe como entrada o vetor de interações $\mathbf{x}_u$ do usuário e o transforma em uma representação latente probabilística. Diferentemente de um autoencoder tradicional, que gera diretamente um vetor latente fixo, o Mult-VAE aprende os parâmetros de uma distribuição de probabilidade no espaço latente.

Formalmente, o codificador é representado por uma função $g_\phi(\cdot)$, parametrizada por uma rede neural, que produz dois vetores: a média $\boldsymbol{\mu}_\phi(\mathbf{x}_u)$ e o desvio padrão $\boldsymbol{\sigma}_\phi(\mathbf{x}_u)$. Esses vetores definem a distribuição aproximada da representação latente do usuário:

$$q_\phi(\mathbf{z}_u | \mathbf{x}_u) = \mathcal{N}(\boldsymbol{\mu}_\phi(\mathbf{x}_u), \text{diag}\{\boldsymbol{\sigma}_\phi^2(\mathbf{x}_u)\}), \quad (2.6)$$

em que $\mathbf{z}_u$ representa o vetor latente do usuário u , com dimensionalidade K . Durante o treinamento, $\mathbf{z}_u$ é amostrado a partir dessa distribuição, permitindo que o modelo represente incertezas associadas às preferências do usuário. Para que essa amostragem seja compatível com o treinamento por retropropagação, utiliza-se o truque da reparametrização:

$$\mathbf{z}_u = \boldsymbol{\mu}_\phi(\mathbf{x}_u) + \boldsymbol{\epsilon} \odot \boldsymbol{\sigma}_\phi(\mathbf{x}_u), \quad \boldsymbol{\epsilon} \sim \mathcal{N}(0, I_K). \quad (2.7)$$

Dessa forma, o codificador aprende a mapear o histórico de interações de cada usuário para uma região do espaço latente, em vez de produzir apenas uma representação determinística. Essa representação será utilizada pelo decodificador para reconstruir as preferências do usuário sobre os itens.

2.1.2.3 Decodificador

O decodificador recebe a representação latente $\mathbf{z}_u$, produzida a partir do codificador, e a utiliza para reconstruir o vetor de interações do usuário. Esse componente é representado por uma função $f_\theta(\cdot)$, parametrizada por uma rede neural, que gera um vetor de pontuações para todos os itens do conjunto de dados:

$$\mathbf{s}_u = f_\theta(\mathbf{z}_u), \quad (2.8)$$

em que $\mathbf{s}_u \in \mathbb{R}^I$ representa as pontuações atribuídas aos I itens para o usuário u . No modelo original, essas pontuações são normalizadas por uma função *softmax*, produzindo uma distribuição de probabilidade sobre os itens:

$$\boldsymbol{\pi}(\mathbf{z}_u) = \text{softmax}(f_\theta(\mathbf{z}_u)). \quad (2.9)$$

Dessa forma, cada elemento $\pi_i(\mathbf{z}_u)$ indica a probabilidade atribuída ao item i para o usuário u . A reconstrução das interações consiste em fazer com que o modelo atribua maiores pontuações aos itens com os quais o usuário interagiu, permitindo estimar quais itens tendem a ser mais relevantes para ele.

No processo de recomendação, os itens podem ser ordenados a partir das pontuações produzidas pelo decodificador. Assim, após o treinamento, o modelo utiliza o histórico observado de um usuário para gerar estimativas de preferência para os demais itens.

2.1.2.4 Função objetivo e treinamento

O treinamento busca ajustar os parâmetros do codificador $g_\phi(\cdot)$ e do decodificador $f_\theta(\cdot)$ de modo que o modelo consiga reconstruir o vetor de interações do usuário a partir da representação latente aprendida. Para isso, o modelo maximiza uma função objetivo baseada na *Evidence Lower Bound* (ELBO), definida por:

$$\mathcal{L}_\beta(\mathbf{x}_u; \theta, \phi) = \mathbb{E}_{q_\phi(\mathbf{z}_u | \mathbf{x}_u)} [\log p_\theta(\mathbf{x}_u | \mathbf{z}_u)] - \beta \cdot \text{KL}(q_\phi(\mathbf{z}_u | \mathbf{x}_u) \| p(\mathbf{z}_u)) \quad (2.10)$$

O primeiro termo da função objetivo está associado à reconstrução das interações do usuário, isto é, mede o quanto o decodificador consegue reconstruir $\mathbf{x}_u$ a partir de $\mathbf{z}_u$. Já o segundo termo corresponde à divergência de Kullback-Leibler, responsável por regularizar a distribuição latente aproximada $q_\phi(\mathbf{z}_u | \mathbf{x}_u)$, aproximando-a da distribuição a priori $p(\mathbf{z}_u)$, geralmente definida como uma normal padrão. O parâmetro β permite controlar o equilíbrio entre reconstruir bem as interações observadas e manter o espaço latente regularizado. Quando β é menor, o modelo dá maior importância à reconstrução; quando β é maior, a regularização sobre o espaço latente se torna mais forte.

Durante o treinamento, o modelo utiliza amostras de usuários e atualiza os parâmetros θ e ϕ por meio de métodos baseados em gradiente. Além disso, o truque da reparametrização permite que a amostragem da variável latente $\mathbf{z}_u$ seja incorporada ao processo de retropropagação, tornando possível treinar o codificador e o decodificador de forma conjunta.

2.1.2.5 Predição e adaptação ao problema deste trabalho

Após o treinamento, o modelo realiza predições a partir do histórico de interações de cada usuário. Dado o vetor $\mathbf{x}_u$, o codificador produz a distribuição latente aproximada do usuário e, no momento da predição, utiliza-se a média dessa distribuição como representação latente:

$$\mathbf{z}_u = \boldsymbol{\mu}_\phi(\mathbf{x}_u) \quad (2.11)$$

Em seguida, essa representação é passada para o decodificador, que gera um vetor de pontuações para os itens:

$$\hat{\mathbf{x}}_u = f_\theta(\mathbf{z}_u) \quad (2.12)$$

Cada elemento $\hat{x}_{ui}$ representa a estimativa de preferência do usuário u em relação ao item i . Assim, os itens podem ser ordenados de acordo com essas pontuações, permitindo selecionar aqueles com maior probabilidade de serem relevantes para o usuário.

No artigo original, o Mult-VAE é utilizado principalmente em um cenário de feedback implícito, no qual o objetivo é gerar rankings de recomendação. Neste trabalho, entretanto, as pontuações produzidas pelo modelo serão utilizadas para compor a matriz estimada $\hat{X}$, necessária para o cálculo das métricas de qualidade preditiva e de justiça de grupo. Quando necessário, essas estimativas serão ajustadas para a escala de avaliação adotada nos conjuntos de dados, de modo que possam ser comparadas com os valores observados da matriz X .

2.2 Algoritmos de Clusterização

Os algoritmos de clusterização são técnicas de aprendizado não supervisionado usadas para organizar dados em grupos de acordo com sua similaridade. Neste trabalho, eles serão utilizados para identificar grupos latentes de usuários e avaliar se diferentes agrupamentos apresentam desigualdades na qualidade das recomendações recebidas.

2.2.1 K-means

O K-Means é um algoritmo de clusterização particional utilizado para dividir um conjunto de dados em k grupos, de modo que elementos semelhantes sejam atribuídos ao mesmo agrupamento. A ideia central do método é representar cada grupo por um centroide e associar cada observação ao centroide mais próximo. Dessa forma, o algoritmo busca formar grupos compactos, nos quais os elementos apresentam menor distância em relação ao centroide do próprio grupo.

De forma geral, o algoritmo opera de maneira iterativa. Inicialmente, são definidos k centroides, que podem ser escolhidos aleatoriamente ou por algum método de inicialização. Em seguida, cada elemento é atribuído ao grupo cujo centroide apresenta a menor distância, geralmente calculada pela distância euclidiana. Após essa etapa, os centroides são atualizados a partir da média dos elementos pertencentes a cada grupo. Esse processo de atribuição e atualização é repetido até que os centroides se estabilizem ou até que um critério de parada seja atingido.

O funcionamento do K-Means está associado ao algoritmo de Lloyd, que descreve um procedimento iterativo baseado na minimização de erro quadrático. Nesse processo, a solução alterna entre duas etapas principais: a definição das regiões associadas aos representantes mais próximos e a atualização desses representantes a partir do centro de massa dos elementos atribuídos a cada região (LLOYD, 1982).

2.2.2 Agrupamento Hierárquico

O agrupamento hierárquico é uma técnica de clusterização que organiza os dados em uma estrutura hierárquica de grupos. Diferentemente de métodos particionais, como o K-Means, esse tipo de abordagem não produz apenas uma divisão fixa dos dados, mas uma sequência de agrupamentos em diferentes níveis de granularidade. No caso do agrupamento hierárquico aglomerativo, cada elemento começa como um grupo individual e, a cada etapa, os grupos mais semelhantes são unidos até que todos os elementos pertençam a um único grupo.

A cada iteração, o algoritmo avalia quais grupos devem ser combinados de acordo com um critério de ligação. No método de Ward, essa escolha é feita buscando minimizar o aumento da variância interna dos grupos após cada fusão. Assim, a união selecionada é aquela que gera a menor perda de homogeneidade dentro dos agrupamentos, favorecendo a formação de grupos relativamente compactos e internamente semelhantes (WARD, 1963).

Ao final do processo, obtém-se uma estrutura hierárquica que permite analisar diferentes quantidades de grupos. A definição do número final de agrupamentos pode ser feita por critérios externos ou por métricas de qualidade da clusterização, como o coeficiente de silhueta.

2.3 Justiça Individual e de Grupo

Todas as métricas de justiça utilizadas neste trabalho foram baseadas no trabalho do (RASTEGARPANAH; GUMMADI; CROVELLA, 2019) para nos dar uma base sólida de métricas de justiça em sistemas de recomendação. As métricas são citadas na subseção 1.3.5 da metodologia.

Vamos começar apresentando a definição do problema. Suponha que $X \in \mathbb{R}^{n \times m}$ é uma matriz de avaliações parcialmente observada de n usuários e m itens, de modo que cada elemento x_{ij} representa uma avaliação dada ao item j por um usuário i . Seja Ω o conjunto de índices de avaliações conhecidas em X . Além disso, Ω_i denota os índices de avaliações de itens conhecidas do usuário i , e Ω_j denota os índices de avaliações de usuários conhecidas para o item j .

Para uma matriz A , $P_\Omega(A)$ é uma matriz em que os elementos $(i, j) \in \Omega$ são a_{ij} e

zero em qualquer outro lugar. De maneira análoga, para um vetor a , $P_{\Omega_j}(a)$ é um vetor cujos elementos $i \in \Omega_j$ são os elementos correspondentes de a e zero para qualquer outra posição. Para este trabalho, nós denotamos a coluna j de A pelo vetor a_j , e a linha i de A pelo vetor a^i . Todos os vetores são vetores coluna.

Dado um sistema de recomendação tradicional, uma matriz de recomendação estimada $\hat{X} = [\hat{X}_{ij}]_{n \times m}$ é gerada. Para este problema de recomendação, assume-se que usuários são um conjunto $\{u_1, u_2, \dots, u_n\}$ e itens são um conjunto $\{v_1, v_2, \dots, v_m\}$.

2.3.1 Justiça Individual

Para cada usuário i , definimos ℓ_i , a perda do usuário i , como o erro quadrático médio estimado sobre as avaliações conhecidas do usuário i . A injustiça individual R_{indv} é definida como a variação das perdas dos usuários.

$$\ell_i = \frac{\|P_{\Omega^i}(\hat{\mathbf{x}}^i - \mathbf{x}^i)\|_2^2}{|\Omega^i|} \quad (2.13)$$

$$R_{indv}(X, \hat{X}) = \frac{1}{n^2} \sum_{k=1}^n \sum_{l>k} (\ell_k - \ell_l)^2 \quad (2.14)$$

2.3.2 Justiça de Grupo

Para cada grupo g de usuários, definimos a perda do grupo ℓ_g , calculada como o erro quadrático médio estimado sobre as avaliações conhecidas dos usuários do grupo g . A medida de justiça social R_{grp} é definida como a variação das perdas dos grupos. Seja G o número de grupos, em que cada grupo é indexado por g . Então, temos:

$$L_i = \frac{\|P_{\Omega_{G_i}}(\hat{X} - X)\|_2^2}{|G_i|} \quad (2.15)$$

$$R_{grp}(X, \hat{X}, G) = \frac{1}{g^2} \sum_{k=1}^g \sum_{l>k} (L_k - L_l)^2 \quad (2.16)$$

2.4 Tecnologias Utilizadas

2.4.1 Python

A linguagem de programação Python ([ROSSUM; DRAKE, 2009](#)) será adotada como a linguagem base para a condução, integração e execução de todos os experimentos deste trabalho. Sua escolha justifica-se por ser a linguagem nativa na qual frameworks essenciais para esta pesquisa, como PyTorch e RecBole, foram desenvolvidos e são majoritariamente

suportados, garantindo total compatibilidade e interoperabilidade entre as ferramentas utilizadas.

Além disso, o Python possui um consolidado ecossistema voltado à ciência de dados e ao aprendizado de máquina, sendo amplamente adotado pela comunidade científica. No contexto deste estudo, a linguagem oferecerá o suporte necessário para a orquestração dos scripts de treinamento dos algoritmos de recomendação, bem como facilitará as etapas de manipulação, pré-processamento das bases de dados e a automação da coleta de métricas de avaliação.

2.4.2 PyTorch

O PyTorch ([PASZKE et al., 2019](#)) será utilizado como biblioteca principal para a implementação e treinamento dos modelos neurais de recomendação adotados neste trabalho. Sua escolha se justifica pelo suporte à construção de modelos de aprendizado profundo de forma flexível, permitindo a definição de arquiteturas customizadas, funções de perda específicas e rotinas de treinamento adaptadas ao problema investigado.

Além disso, o PyTorch oferece recursos como diferenciação automática, operações com tensores e execução acelerada em GPU, o que facilita a experimentação com métodos neurais e torna viável a adaptação dos modelos para incorporar a avaliação de perdas por grupos de usuários.

2.4.3 RecBole

A biblioteca RecBole ([ZHAO et al., 2021](#)) será empregada como base experimental para a implementação, treinamento e avaliação dos algoritmos de recomendação. Trata-se de um framework desenvolvido em Python e PyTorch voltado à pesquisa em sistemas de recomendação, oferecendo uma estrutura unificada para processamento de dados, configuração de modelos, treinamento, avaliação e comparação de resultados.

Sua utilização contribui para a padronização dos experimentos, reduzindo a necessidade de implementar do zero etapas recorrentes, como leitura dos datasets, divisão dos dados, geração de batches e cálculo de métricas. No contexto deste trabalho, a RecBole também será utilizado como ponto de partida para adaptações nos modelos neurais.

Referências

- BERNARDINO, G. S.; GONÇALVES, A. L. A education profile model applied in the context of recommender systems. *IEEE Latin America Transactions*, v. 17, n. 3, p. 505–512, 2019. Citado na página 2.
- BEUTEL, A. et al. Beyond globally optimal: Focused learning for improved recommendations. In: *Proceedings of the 26th International Conference on World Wide Web*. [S.l.: s.n.], 2017. p. 203–212. Citado na página 2.
- BURKE, R.; SONBOLI, N.; ORDONEZ-GAUGER, A. Balanced neighborhoods for multi-sided fairness in recommendation. In: FRIEDLER, S. A.; WILSON, C. (Ed.). *Proceedings of the 1st Conference on Fairness, Accountability and Transparency*. PMLR, 2018. (Proceedings of Machine Learning Research, v. 81), p. 202–214. Disponível em: <<https://proceedings.mlr.press/v81/burke18a.html>>. Citado na página 2.
- CAVALCANTE, C. G. S.; FETTERMAN, D. C. Recommendations for product development of intelligent products. *IEEE Latin America Transactions*, v. 17, n. 10, p. 1645–1652, Dec. 2019. Disponível em: <<https://latamt.ieeer9.org/index.php/transactions/article/view/631>>. Citado na página 2.
- CELMA, O. *Music Recommendation and Discovery: The Long Tail, Long Fail, and Long Play in the Digital Music Space*. [S.l.]: Springer Berlin Heidelberg, 2010. ISBN 978-3-642-13286-5. Citado na página 4.
- DANDEKAR, P.; GOEL, A.; LEE, D. T. Biased assimilation, homophily, and the dynamics of polarization. *Proceedings of the National Academy of Sciences*, v. 110, n. 15, p. 5791–5796, 2013. Disponível em: <<https://www.pnas.org/doi/abs/10.1073/pnas.1217220110>>. Citado na página 2.
- DATTA, A.; TSCHANTZ, M. C.; DATTA, A. *Automated Experiments on Ad Privacy Settings: A Tale of Opacity, Choice, and Discrimination*. 2015. Disponível em: <<https://arxiv.org/abs/1408.6491>>. Citado na página 3.
- GLOROT, X.; BORDES, A.; BENGIO, Y. Deep sparse rectifier neural networks. In: GORDON, G.; DUNSON, D.; DUDÍK, M. (Ed.). *Proceedings of the Fourteenth International Conference on Artificial Intelligence and Statistics*. Fort Lauderdale, FL, USA: PMLR, 2011. (Proceedings of Machine Learning Research, v. 15), p. 315–323. Disponível em: <<https://proceedings.mlr.press/v15/glorot11a.html>>. Citado na página 10.
- HAZRATI, N.; RICCI, F. Choice models and recommender systems effects on users' choices. *User Modeling and User-Adapted Interaction*, v. 34, n. 1, p. 109–145, mar. 2024. ISSN 1573-1391. Disponível em: <<https://doi.org/10.1007/s11257-023-09366-x>>. Citado na página 2.
- HE, X. et al. *Neural Collaborative Filtering*. 2017. Disponível em: <<https://arxiv.org/abs/1708.05031>>. Citado 2 vezes nas páginas 8 e 9.

- LIANG, D. et al. *Variational Autoencoders for Collaborative Filtering*. 2018. Disponível em: <<https://arxiv.org/abs/1802.05814>>. Citado na página 12.
- LLOYD, S. P. Least squares quantization in pcm. *IEEE Transactions on Information Theory*, v. 28, n. 2, p. 129–137, 1982. Citado na página 16.
- PASZKE, A. et al. Pytorch: An imperative style, high-performance deep learning library. In: WALLACH, H. et al. (Ed.). *Advances in Neural Information Processing Systems*. Curran Associates, Inc., 2019. v. 32. Disponível em: <https://proceedings.neurips.cc/paper_files/paper/2019/file/bdbca288fee7f92f2bfa9f7012727740-Paper.pdf>. Citado na página 18.
- PEREIRA, F. S. F. et al. That’s my jam! uma análise temporal sobre a evolução das preferências dos usuários em uma rede social de músicas. In: *Anais do VII Brazilian Workshop on Social Network Analysis and Mining*. Porto Alegre, RS, Brasil: SBC, 2018. p. 145–156. ISSN 2595-6094. Disponível em: <<https://sol.sbc.org.br/index.php/brasnam/article/view/3587>>. Citado na página 2.
- RASTEGARPANAH, B.; GUMMADI, K. P.; CROVELLA, M. Fighting fire with fire: Using antidote data to improve polarization and fairness of recommender systems. In: *Proceedings of the Twelfth ACM International Conference on Web Search and Data Mining*. New York, NY, USA: Association for Computing Machinery, 2019. (WSDM ’19), p. 231–239. ISBN 9781450359405. Disponível em: <<https://doi.org/10.1145/3289600.3291002>>. Citado na página 16.
- RICCI, F.; ROKACH, L.; SHAPIRA, B. *Recommender Systems Handbook*. 3rd. ed. [S.l.]: Springer, 2022. Citado na página 2.
- ROSSUM, G. V.; DRAKE, F. L. *Python 3 Reference Manual*. Scotts Valley, CA: CreateSpace, 2009. Citado na página 17.
- SANTOS, R. V. M. d.; COMARELA, G. V. Group fairness in recommendation systems: The importance of hierarchical clustering in identifying latent groups in movielens and amazon books. *Journal on Interactive Systems*, v. 16, n. 1, p. 505–515, Jul. 2025. Disponível em: <<https://journals-sol.sbc.org.br/index.php/jis/article/view/5407>>. Citado 2 vezes nas páginas 2 e 3.
- SOCHER, R. et al. Reasoning with neural tensor networks for knowledge base completion. In: BURGESS, C. et al. (Ed.). *Advances in Neural Information Processing Systems*. Curran Associates, Inc., 2013. v. 26. Disponível em: <https://proceedings.neurips.cc/paper_files/paper/2013/file/b337e84de8752b27eda3a12363109e80-Paper.pdf>. Citado na página 11.
- WARD, J. H. Hierarchical grouping to optimize an objective function. *Journal of the American Statistical Association*, v. 58, n. 301, p. 236–244, 1963. Citado na página 16.
- ZHAO, W. X. et al. *RecBole: Towards a Unified, Comprehensive and Efficient Framework for Recommendation Algorithms*. 2021. Disponível em: <<https://arxiv.org/abs/2011.01731>>. Citado na página 18.